

6.(라즈베리파이)

Flask 및 센서제어

IoT 사물인터넷

학습목표

- Flask를 설치하고 웹 서버를 구축할 수 있다.
- Flask를 활용하여 센서를 제어할 수 있다.
- IP, Port, Process, Cron, Daemon에 대해서 이해할 수 있다.

1. Flask 세팅하기

1. Flask

- 1) 파이썬 언어를 이용하여 웹을 개발할 수 있게 해주는 웹개발 프레임워크
 - 프레임워크란 쉽게 개발할 수 있도록 여러 개의 라이브러리, 모듈 등을 묶어서 제공하는 기능



2) 풀스택 프레임워크 장고(Django)

- : 웹 프로그래밍을 할 때 필요로 하는 모든 것들을 종합적으로 갖춘 프레임워크
- : 인증과 권한, ORM (Object Relational Mapping) 객체와 데이터베이스의 관계를 매핑해주는 도구, 템플릿 라이브러리, 로그처리, 보안, 관리자 기능 등 다양한 요소가 기본적으로 제공

1. Flask 세팅하기

2. Flask 설치하기

최신 버전의 라즈비안에는 이미 Flask가 설치되어 있음.

설치 확인하기

: `$ pip3 show flask`

```
pi@raspberrypi:~/webapps $ pip3 show flask
Name: Flask
Version: 1.0.2
Summary: A simple framework for building complex web applications.
Home-page: https://www.palletsprojects.com/p/flask/
Author: Armin Ronacher
Author-email: armin.ronacher@active-4.com
License: BSD
Location: /usr/lib/python3/dist-packages
Requires:
Required-by:
pi@raspberrypi:~/webapps $
```

설치가 필요할 경우

: `$ sudo pip3 install flask`

2. Flask 웹서버 구축하기

1. Flask 실행하기

1) 폴더 및 파일 구성

: webapps - ch06 - app_start.py

2) 파이썬 코드 작성

```
ch06 > helloworld > app_start.py
1  #flask모듈에서 Flask 클래스 import
2  from flask import Flask
3
4  #flask객체를 생성하는 코드
5  #__name__은 자신의 파일명을 의미
6  #즉 자신의 파일을 가지고 객체를 생성
7  app = Flask(__name__)
8
```

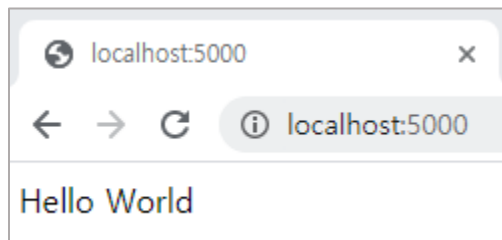
2. Flask 웹서버 구축하기

1. Flask 실행하기

2) 파이썬 코드 작성

return “본인닉네임”

```
9  #라우팅을 위한 뷰 함수 등록 코드
10 #클라이언트가 URI로 "/"를 요청하면 아래있는 뷰함수가 자동 호출
11 #http://localhost:5000/
12 @app.route("/")
13 def helloworld():
14     return "Hello World"
15     #뷰함수에는 반드시 return문이 있어야 한다.
16
17 #app_start.py파일을 직접 실행시켰을때 __main__이라는 이름을 가질 수 있음
18 #app.run()이 호출되면 웹서버가 동작
19 #즉, 파일을 직접 실행시켰을 때만 "서버를 작동시켜"라는 뜻
20 if __name__=="__main__":
21     app.run(host="0.0.0.0") port=5001
22 # 0.0.0.0은 어떤 ip에서 요청이 들어와도 응답을 하겠다는 의미
```



http://192.168.0.2xx:5000/ (라즈베리파이 IP)

Flask 기본 포트 5000

2. Flask 웹서버 구축하기

<http://192.168.0.220:5000/>

노경록 192.168.0.220	김조섭 221	소현철 222
김유석 223	박규민 224	이채연 225
홍현지 226	유민우 227	안예진 228
홍현표 229	이동욱 230	유연욱 231
심희준 232	최승만 233	김대희 234
김태민 235	윤성현 236	강지우 237
이민영 238	염지현 239	김한별 240
박승범 241	고창훈 242	

3. Flask 라우팅

1. Flask에서 URL처리 방법

1) Flask에서 URL을 처리

: URI 디스패치 - 사용자가 입력한 URI를 지켜보고 분석하여 올바른 길로 안내

: 뷰 데코레이션을 통해서 뷰 함수를 생성하여 올바른 길로 안내

: 즉, Flask 라우팅이란 Flask웹서버를 활용, 전송경로를 지정해서 올바른 길로 안내하는 것.

2) 폴더 및 파일 구성

: webapps - ch06 - app_start2.py

3. Flask 라우팅

1. Flask에서 URL처리 방법

3) 파이썬 코드 작성

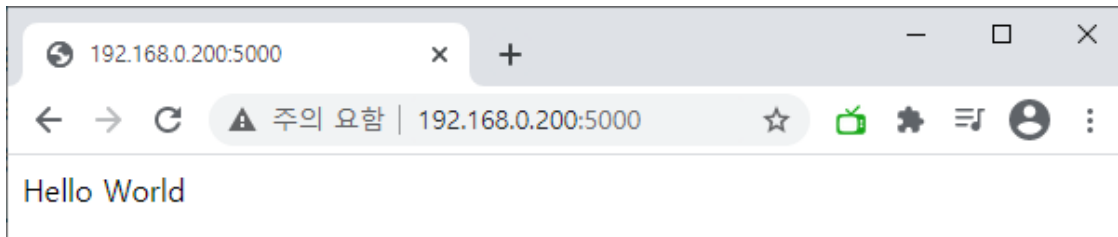
```
1  from flask import Flask  #flask모듈에서 Flask 클래스 import
2
3  #자기 자신의 파일명으로 객체 생성
4  app = Flask(__name__)
5
6  #클라이언트가 URI로 "/"를 요청하면 아래있는 뷰함수가 자동 호출
7  @app.route("/")          #http://localhost:5000/
8  def helloworld():
9      return "Hello World"
10
11 @app.route("/led/on")      #http://localhost:5000/led/on
12 def led_on():
13     return "LED ON"
14
15 @app.route("/led/off")     #http://localhost:5000/led/off
16 def led_off():
17     return "LED OFF"
18
19 #자신의 파일을 직접 실행시켜 app.run()를 호출, 서버 실행
20 if __name__ == "__main__":
21     app.run(host="0.0.0.0")
```

3. Flask 라우팅

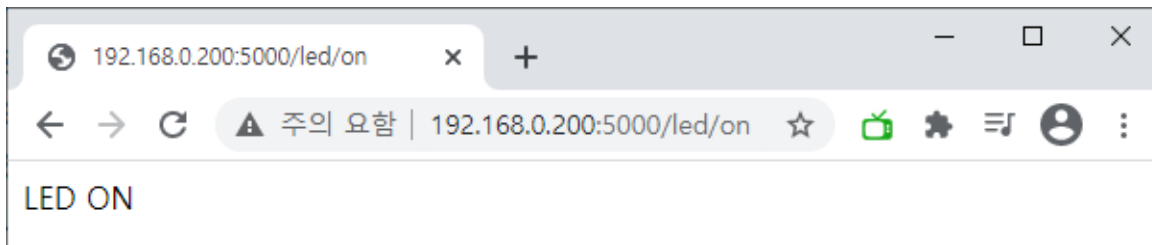
1. Flask에서 URL처리 방법

4) 웹브라우저에서 실행

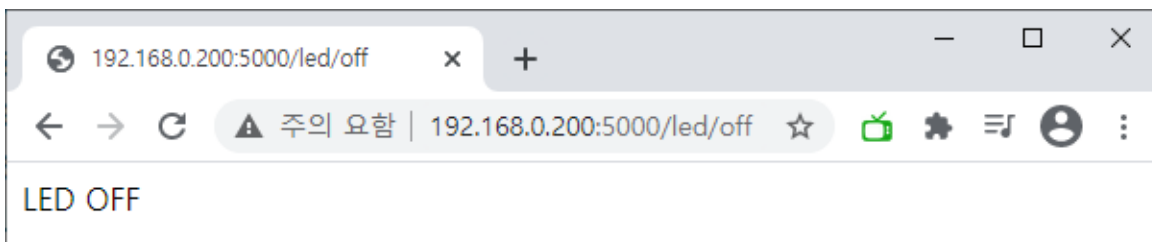
: <http://localhost:5000/>



: <http://localhost:5000/led/on>



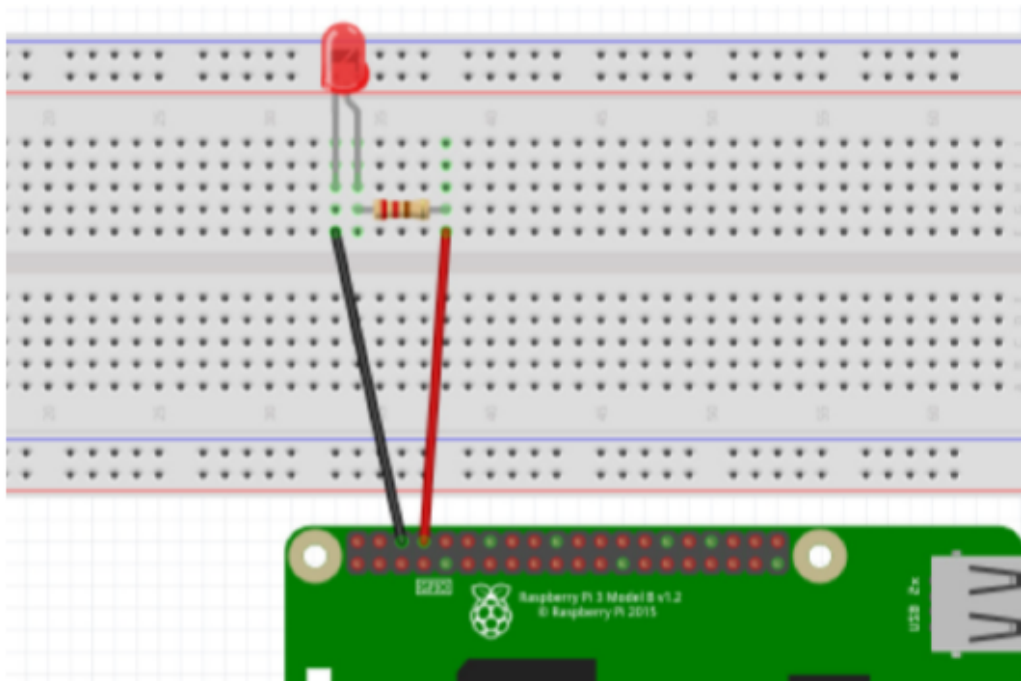
: <http://localhost:5000/led/off>



4. Flask LED 제어하기

1. Flask를 이용한 LED 실습 (정적 라우팅)

1) 브레드보드 연결하기 (저항 220Ω, LED)



RP 4	LED
8번핀	+
6번핀(GND)	-

4. Flask LED 제어하기

1. Flask를 이용한 LED 실습 (정적 라우팅)

2) 파이썬 코드 작성하기 (webapps - ch06 - app_start3.py)

```
1  from flask import Flask  #flask모듈에서 Flask 클래스 import
2  import RPi.GPIO as GPIO
3
4  #자기 자신의 파일명으로 객체 생성
5  app = Flask(__name__)
6
7  LED = 8
8  GPIO.setmode(GPIO.BOARD) #BOARD는 커넥터 pin번호 사용
9  GPIO.setup(LED, GPIO.OUT, initial=GPIO.LOW)
10
11 #클라이언트가 URI로 "/"를 요청하면 아래있는 뷰함수가 자동 호출
12 @app.route("/")          #http://localhost:5000/
13 def helloworld():
14     return "Hello World"
15
16 @app.route("/led/on")     #http://localhost:5000/led/on
17 def led_on():
18     GPIO.output(LED, GPIO.HIGH)
19     return "LED ON"
20
```

4. Flask LED 제어하기

1. Flask를 이용한 LED 실습 (정적 라우팅)

2) 파이썬 코드 작성하기 (webapps - ch06 - app_start3.py)

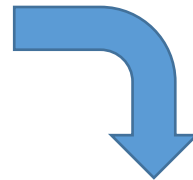
```
20
21 @app.route("/led/off")          #http://localhost:5000/led/off
22 def led_off():
23     GPIO.output(LED, GPIO.LOW)
24     return "LED OFF"
25
26 #GPIO 설정 초기화(모듈 점유 해제)
27 @app.route("/gpio/cleanup")
28 def gpio_cleanup():
29     GPIO.cleanup()
30     return "GPIO CLEANUP"
31
32 #자신의 파일을 직접 실행시켜 app.run()를 호출, 서버 실행
33 if __name__ == "__main__":
34     app.run(host="0.0.0.0")
```

4. Flask LED 제어하기

2. Flask를 이용한 LED 실습 (동적 라우팅 <산형 괄호>)

- 1) 동적으로 변경되는 URL의 뷰함수를 사용하기 위해서는 산호괄호 < > 사용
- 2) 정적 라우팅과 달리 뷰함수의 매개변수로 받아서 실행
- 3) 파이썬 코드 작성하고 실행

```
16 @app.route("/led/on") #http
17 def led_on():
18     GPIO.output(LED, GPIO.HIGH)
19     return "LED ON"
20
21 @app.route("/led/off") #http
22 def led_off():
23     GPIO.output(LED, GPIO.LOW)
24     return "LED OFF"
```



```
@app.route("/led/<state>")
def led(state): #<state>를 매개변수로 받아 실행
    if state == "on":
        GPIO.output(LED, GPIO.HIGH)
    else:
        GPIO.output(LED, GPIO.LOW)
    return "LED "+state
```

4. Flask LED 제어하기

3. Flask를 이용한 LED 실습 (동적 라우팅 <쿼리 스트링>)

- 1) 동적으로 변경되는 URL함수는 다음과 같이 쿼리 스트링 방식으로 호출 가능
- 2) 쿼리 스트링이란 `http://localhost:5000/led?state=on`과 같이 주소를 호출할 때 '?' 뒤에 오는 것을 말함.

: 클라이언트가 쿼리스트링을 웹서버에 요청할 때 추가로 전달하는 데이터

- 3) 전달되는 데이터는 `request` 클래스의 `args` 객체를 사용하여 받을 수 있음.

: `from flask import request`

예시) 구글 검색 URL

`https://www.google.com/search?q=raspberry&hl=ko`

4. Flask LED 제어하기

3. Flask를 이용한 LED 실습 (동적 라우팅 <쿼리 스트링>)

4) 파이썬 코드 작성

```
1  # Flask, request 클래스 import
2  from flask import Flask, request
3  import RPi.GPIO as GPIO
4
5  #자기 자신의 파일명으로 객체 생성
6  app = Flask(__name__)
7
8  LED = 8
9  GPIO.setmode(GPIO.BOARD) #BOARD는 커넥터 pin번호 사용
10 GPIO.setup(LED, GPIO.OUT, initial=GPIO.LOW)
11
12 @app.route("/")
13 def helloworld():
14     return "Hello World"
15
```

4. Flask LED 제어하기

3. Flask를 이용한 LED 실습 (동적 라우팅 <쿼리 스트링>)

4) 파이썬 코드 작성

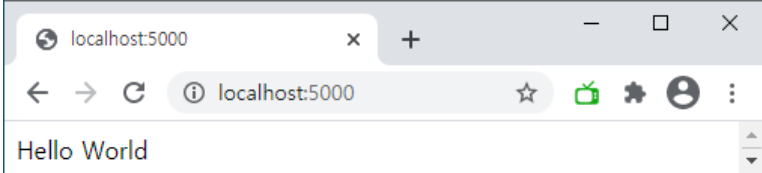
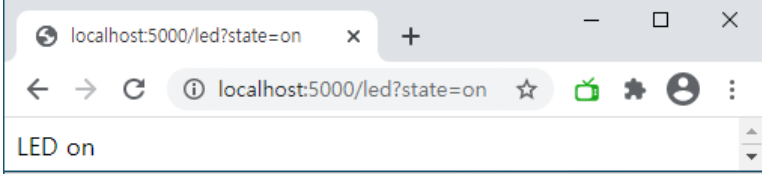
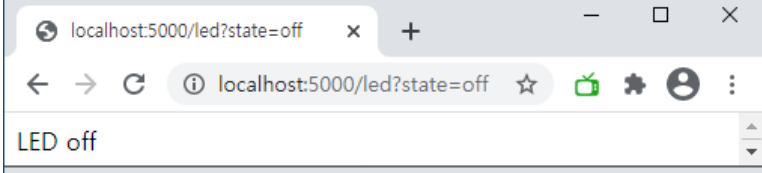
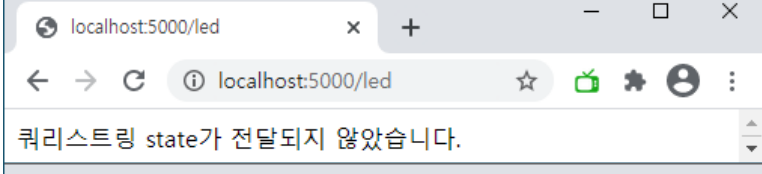
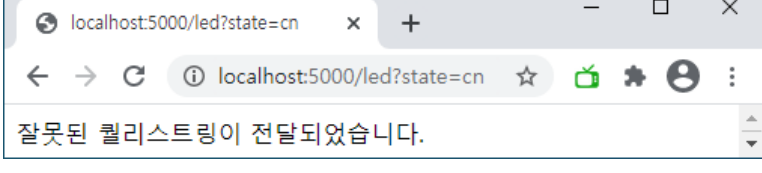
1) `http://localhost:5000/led?state=on`

```
16 @app.route("/led")
17 def led():
18     state = request.args.get("state", "error")
19     if state == "on":
20         GPIO.output(LED, GPIO.HIGH)
21         return "LED "+state
22     elif state == "off":
23         GPIO.output(LED, GPIO.LOW)
24     elif state == "error":
25         return "쿼리스트링 state가 전달되지 않았습니다."
26     else:
27         return "잘못된 쿼리스트링이 전달되었습니다."
28
29     return "LED "+state
30
31 @app.route("/gpio/cleanup")
32 def gpio_cleanup():
33     GPIO.cleanup()
34     return "GPIO CLEANUP"
35
36 if __name__ == "__main__":
37     app.run(host="0.0.0.0", port=6000)
```

4. Flask LED 제어하기

3. Flask를 이용한 LED 실습 (동적 라우팅 <쿼리 스트링>)

5) 웹브라우저에서 실행

- 5)  : localhost:5000
- 6)  : localhost:5000/led?state=on
- 7)  : localhost:5000/led?state=off
- 8)  : localhost:5000/led
- 9)  : localhost:5000/led?state=clear

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

1) 웹페이지를 통해서 라즈베리파이에 연결된 LED를 제어한다.

2) 폴더 및 파일 구성

: webapps - ch06 - home - index.py (파일)

- **static** (폴더)

- templates (폴더)

: home - 최상위 root 폴더

: static - 리소스(자원) 폴더 / css, image, media 등의 파일을 저장

: templates - html 파일 폴더 / flask는 templates폴더에 있는 html을 호출, 폴더명 변경X

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

3) 파이썬 코드 작성 (webapps - ch06 - home - index.py)

```
1  from flask import Flask, request
2  from flask import render_template
3  import RPi.GPIO as GPIO
4
5  app=Flask(__name__)
6  GPIO.setmode(GPIO.BOARD)
7  GPIO.setup(8, GPIO.OUT,initial=GPIO.LOW)
8
9  @app.route("/")
10 def home():
11     return render_template("index.html")
12
```

```
13 @app.route("/led/on")
14 def led_on():
15     try:
16         GPIO.output(8, GPIO.HIGH)
17         return "ok"
18     except expression as identifier:
19         return "fail"
20
21 @app.route("/led/off")
22 def led_off():
23     try:
24         GPIO.output(8, GPIO.LOW)
25         return "ok"
26     except expression as identifier:
27         return "fail"
28
29 if __name__=="__main__":
30     app.run(host="0.0.0.0")
```

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

4) html 코드 작성 (webapps - ch06 - home - templates - index.html)

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5      <title>Home Network</title>
6  </head>
7  <body>
8      <div>
9          <div>
10             <h2>HOME IoT</h2>
11          </div>
12          <div>
13              <div>
14                  <button onclick="led_on()">LED ON</button>
15              </div>
16              <div>
17                  <button onclick="led_off()">LED OFF</button>
18              </div>
19          </div>
20          <div id="result">
21          </div>
22      </div>
```

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

4) html 코드 작성 (webapps - ch06 - home - templates - index.html)

```
23     <script>
24         function led_on(){
25             fetch("/led/on")
26             .then(response=>response.text())
27             .then(data=>{
28                 console.log(data);
29                 let result=document.querySelector("#result");
30                 if(data=="ok"){
31                     result.innerHTML="<h1>LED is runinng</h1>";
32                 } else {
33                     result.innerHTML="<h1>error</h1>";
34                 }
35             });
36     }
```

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

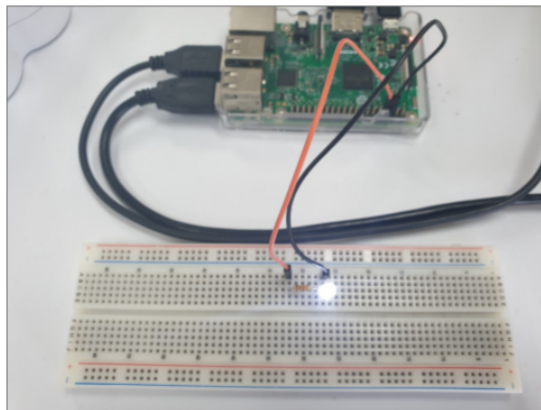
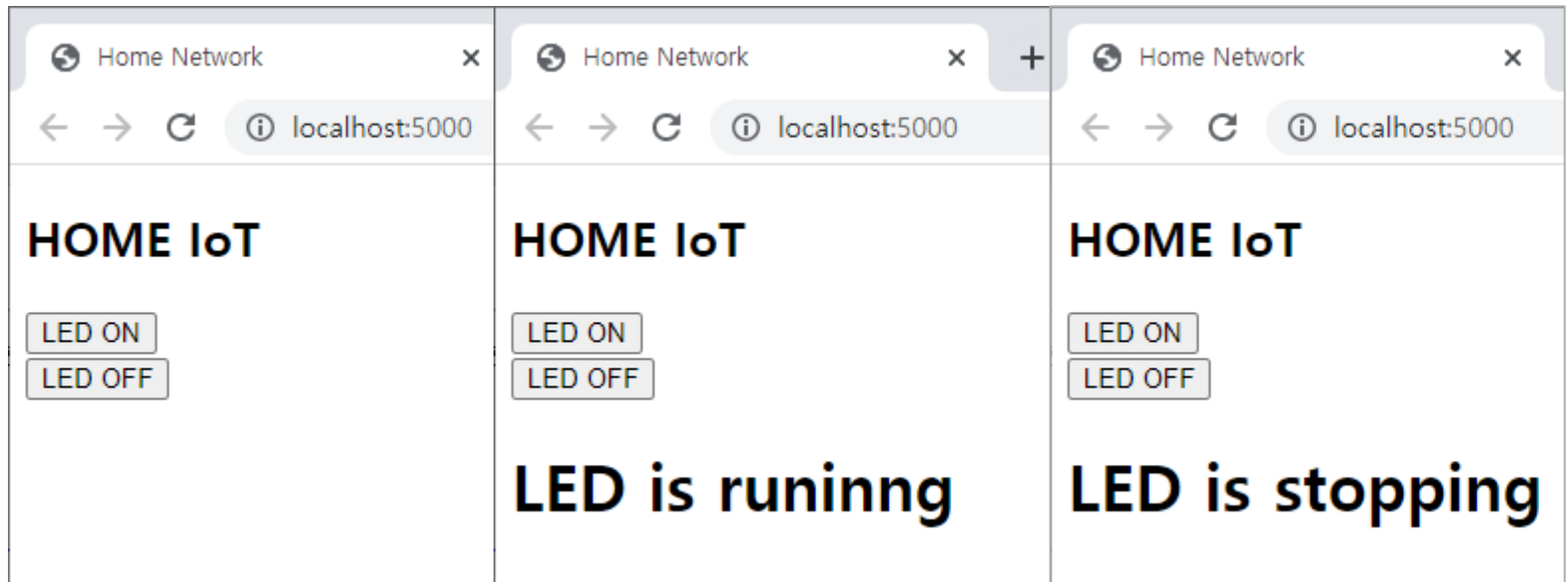
4) html 코드 작성 (webapps - ch06 - home - templates - index.html)

```
37  function led_off(){
38      fetch("/led/off")
39      .then(response=>response.text())
40      .then(data=>{
41          console.log(data);
42          let result=document.querySelector("#result");
43          if(data=="ok"){
44              result.innerHTML("<h1>LED is stopping</h1>");
45          } else {
46              result.innerHTML("<h1>error</h1>");
47          }
48      });
49  }
50  </script>
51  </body>
52  </html>
```

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

5) 웹브라우저에서 실행



5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

6) 자바스크립트 fetch()함수

```
function led_on(){
  fetch("/led/on")
    .then(response=>response.text())
    .then(data=>{
      console.log(data);
      let result=document.querySelector("#result");
      if(data=="ok"){
        result.innerHTML="<h1>LED is runinng</h1>";
      } else {
        result.innerHTML="<h1>error</h1>";
      }
    });
}
```

```
@app.route("/led/on")
def led_on():
    try:
        GPIO.output(8, GPIO.HIGH)
        return "ok"
    except expression as identifier:
        return "fail"
```

- : fetch()를 통해서 index.py의 led_on() 메서드를 호출, 그 결과를 response 통해서 받음
- : led_on메서드에 의해 정상작동하면 "ok"를 리턴 받음.
- : fetch()는 요청시 대기(pending)하고 있다가 요청 결과를 받으면 대기가 끝나고 다음 코드 실행

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

7) 참고 - CSS 적용하기 (webapps - ch06 - home - static - style.css)

```
1  body {  
2      background-color: antiquewhite;  
3  }  
4  
5  .container {  
6      width: 700px;  
7      margin: 0 auto;  
8      text-align: center;  
9  }  
10  
11  .main {  
12      display: flex;  
13  }  
14  
15  .main div {  
16      flex: 1;  
17  }  
18  
19  .main div button {  
20      background-color: rgb(192,114,144);  
21      width: 150px;  
22      height: 80px;  
23      border-radius: 10px;  
24  }
```

5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

7) 참고 - CSS 적용하기 (html파일 수정)

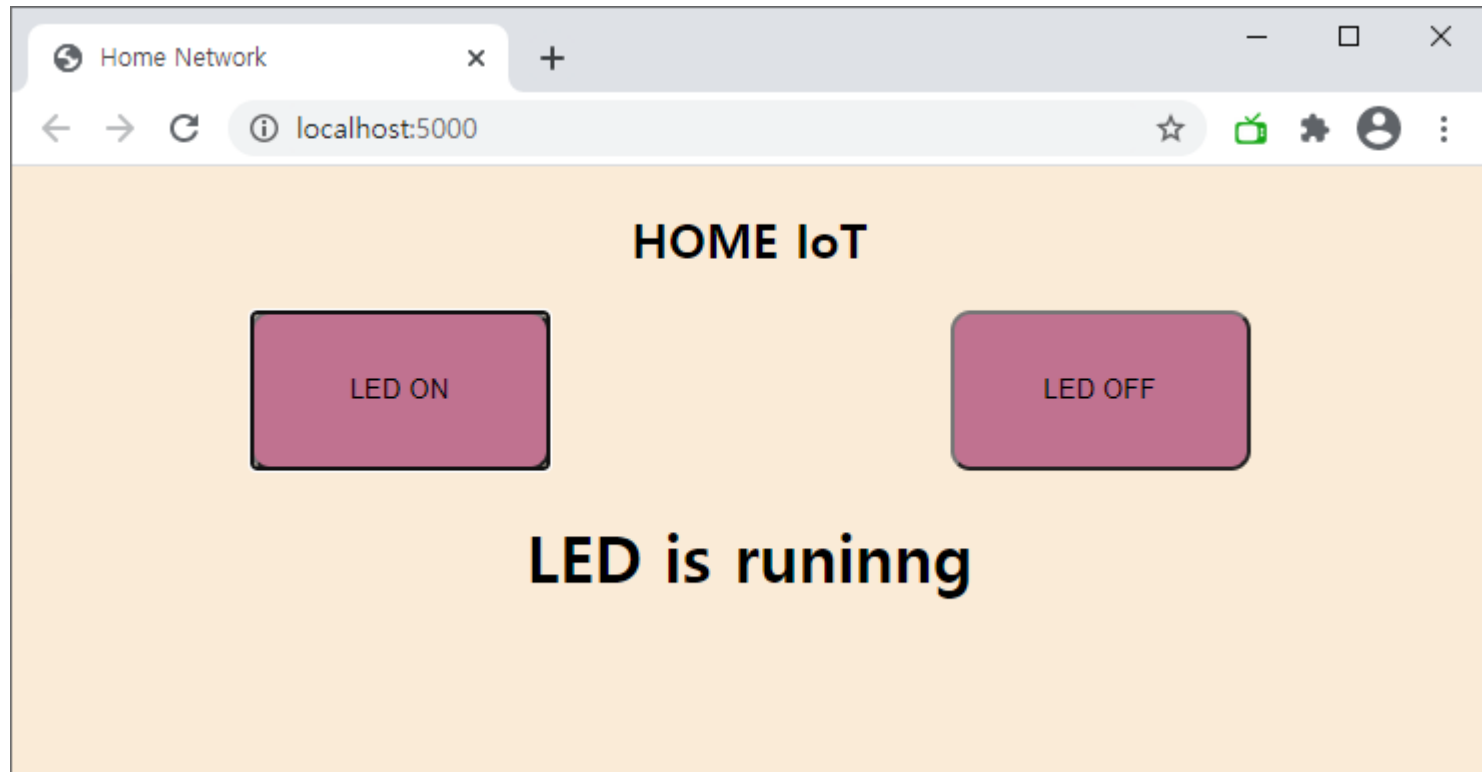
```
3 <head>
4     <meta charset="UTF-8">
5     <title>Home Network</title>
6     <link rel="stylesheet" href="{{url_for('static', filename='style.css')}}">
7 </head>
8 <body>
9     <div class="container">
10         <div class="header">
11             <h2>HOME IoT</h2>
12         </div>
13         <div class="main">
14             <div>
15                 <button onclick="led_on()">LED ON</button>
16             </div>
17             <div>
18                 <button onclick="led_off()">LED OFF</button>
19             </div>
20         </div>
```



5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

7) 참고 - CSS 적용하기 (웹브라우저에서 실행)



5. Flask 웹페이지

1. 웹페이지를 통한 Flask LED 실습

[실습] 큰방 on, 큰방 off (LED 노란색)

거실 on, 거실 off (LED 초록색)

우리집 HOME IoT

큰방 전등 ON

큰방 전등 OFF

거실 전등 ON

거실 전등 OFF

6. Flask 웹페이지 (Live Video Streaming)

1. OpenCV를 활용한 웹페이지 실습 (webapps - ch06 - live - index.py)

```
import cv2
from flask import Flask, render_template, Response

app = Flask(__name__)
# 가중치 파일 경로
cascade_filename = "/home/pi/opencv/data/haarcascades/haarcascade_frontalface_alt.xml"
# 모델 불러오기
cascade = cv2.CascadeClassifier(cascade_filename)
```

6. Flask 웹페이지 (Live Video Streaming)

1. OpenCV를 활용한 웹페이지 실습 (webapps - ch06 - live - index.py)

```
def imgDetector(img, cascade):
    # 영상 압축
    img = cv2.resize(img, dsize=None, fx=1.0, fy=1.0)
    # 그레이 스케일 변환
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # cascade 얼굴 탐지 알고리즘
    results = cascade.detectMultiScale(
        gray, # 입력 이미지
        scaleFactor=1.5, # 이미지 피라미드 스케일 factor
        minNeighbors=5, # 인접 객체 최소 거리 픽셀
        minSize=(20, 20), # 탐지 객체 최소 크기
    )

    for box in results:
        x, y, w, h = box
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 255, 255), thickness=2)

    return img
```

6. Flask 웹페이지 (Live Video Streaming)

1. OpenCV를 활용한 웹페이지 실습

```
def gen():
    camera = cv2.VideoCapture(0)
    camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    while 1:
        _, frame = camera.read()
        retImg = imgDetector(frame, cascade)
        cv2.imwrite("pic.jpg", retImg)

        yield (
            b"--frame\r\n"
            b"Content-Type: image/jpeg\r\n\r\n" + open("pic.jpg", "rb").read() + b"\r\n"
        )

    camera.release()
    cv2.destroyAllWindows()
```

[힌트]

yield

6. Flask 웹페이지 (Live Video Streaming)

1. OpenCV를 활용한 웹페이지 실습

```
@app.route("/")
def index():
    """Video streaming ."""
    return render_template("index.html")

@app.route("/video_feed")
def video_feed():
    """Video streaming route. Put this in the src attribute of an img tag."""
    return Response(gen(), mimetype="multipart/x-mixed-replace; boundary=frame")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=9000, debug=True, threaded=True)
```

6. Flask 웹페이지 (Live Video Streaming)

1. OpenCV를 활용한 웹페이지 실습

(webapps - ch06 - live - templates - index.html)

```
<html>

<head>
    <title>Video Streaming </title>
</head>

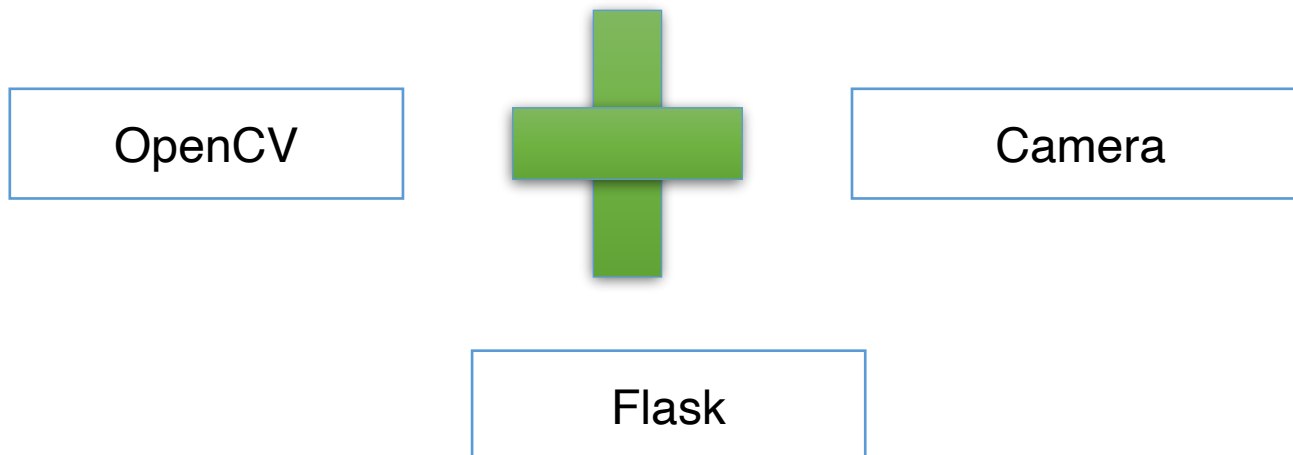
<body>
    <h1> Live Video Streaming </h1>
    
</body>

</html>
```

7. Flask 웹페이지

1. Flask + OpenCV 활용하기 - (3시간)

- 팀별 과제로 구현한 OpenCV 프로젝트를 Flask 웹페이지로 만들어 보자



IoT 무인점포



<https://www.youtube.com/watch?v=WXVb31y0v-Y>

7. Flask 웹페이지

[문제] Flask + 모스부호 웹페이지

모스부호 입력 -> 영어 결과 출력

영어 입력 -> 모스부호 출력

영어 + 모스 부호 혼합 입력 -> 오류 출력



모스 부호

7. Flask 웹페이지

(webapps - ch06 - morse - templates - index.html)

```
<form class="form-signin" action="/morse_decoding" method="POST">

    <h1 class="h3 mb-3 font-weight-normal">모스 부호</h1>
    <label for="inputMorse" class="sr-only">입력해 주세요</label>

    <input type="text" name="inputMorse" id="inputMorse"
           class="form-control" required autofocus>

    <input type="submit" class="btn btn-lg btn-primary btn-block"
           value="해독">

    <div id="result">{{ result }}</div>

</form>
```

7. Flask 웹페이지

(webapps - ch06 - morse - index.py)

```
@app.route("/morse_decoding", methods=["POST", "GET"])
def morse_decoding():
    if request.method == "POST":
        inputMorse = request.form.get("inputMorse")

        # 코드를 넣어주세요.
        english_text = inputMorse

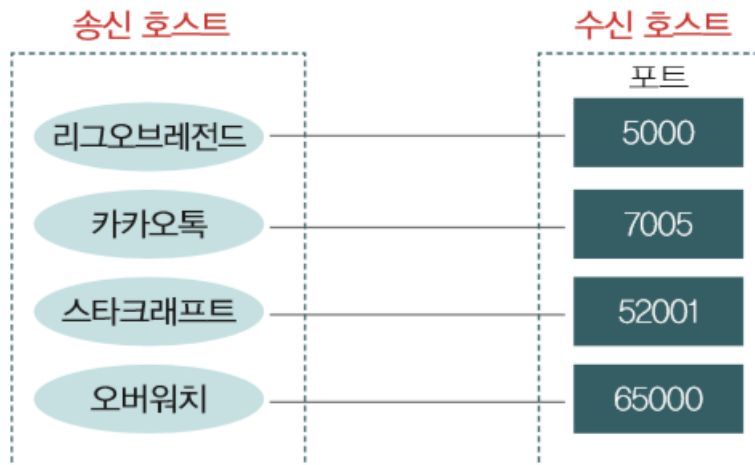
        return render_template("index.html", result=english_text)
```

6. Port, Process 개념잡기

1. Port

컴퓨터 간 상호 통신을 위해 프로토콜에서 이용하는 가상의 연결단을 의미

- : 물리적으로 큰 공간을 논리적으로 나눠서 사용
- : 하나의 포트를 여러 개의 프로그램이 중복해서 점유하고 사용할 수 없음
- : 수신 호스트가 4개의 컴퓨터와 서로 다른 통신을 한다는 의미는 최소 4개 이상의 Port로 통신을 하고 있다는 의미



VNC

5800

SSH

22

6. Port, Process 개념잡기

1. Port

2) Well-Known Port

- : 0번 ~ 1023번 - Well-Known port, 잘 알려진 포트
- : 1024번 ~ 49151번 - registered port, 등록된 포트
- : 49152번 ~ 65535번 - dynamic port, 동적 포트
- : 잘 알려진 포트번호의 대표적인 예

- 20 : FTP(data)
- 21 : FTP(제어)
- 22 : SSH
- 23 : 텔넷
- 53 : DNS
- 80 : 월드 와이드 웹 HTTP
- 119 : NNTP
- 443 : TLS/SSL 방식의 HTTP

6. Port, Process 개념잡기

2. Process

1) 프로세스란 실행중인 프로그램

: 프로그램은 실행되기를 기다리는 명령어(코드)와 정적인 데이터의 묶음

: 이 프로그램이 메모리에 적재되면 생명이 있는 프로세스가 됨

2) 프로세스 테스트 하기

: webapps - ch06 - app_start.py

: 실행

\$ cd ~/webapps/ch06

\$ python3 app_start.py

: 웹브라우저에서 실행여부 확인

: 새로운 터미널을 열고 포그라운드 프로세스 확인 (-f : 자세히보기)

\$ ps -f

```
pi@raspberrypi:~ $ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
pi           13939   13938  1 19:00 pts/3    00:00:00 -bash
pi           13953   13939  0 19:00 pts/3    00:00:00 ps -f
```

: 현재 실행되고 있는 app_start.py 는 백그라운드에서 실행되어 보이지 않음.

6. Port, Process 개념잡기

2. Process

3) 프로세스 테스트 하기

모든 프로세스에서 app_start.py 포함하는 결과 표시

: `$ ps -ef | grep app_start.py`

: pid는 프로세스의 고유한 id를 의미. 중복되지 않는 유일한 값

```
• (base) kelly@kellysui-noteubug fastrun % ps -ef | grep app_start.py
    501 10515  9211    0  8:56AM ttys000      0:00.19 python app_start.py
    501 10668 10524    0  8:57AM ttys002      0:00.00 grep app_start.py
```

: 프로세스 종료하기

`$ sudo kill 10515`

`$ ps -ef | grep app_start.py`

```
• (base) kelly@kellysui-noteubug fastrun % ps -ef | grep app_start.py
    501 10860 10524    0  8:59AM ttys002      0:00.00 grep app_start.py
```

6. Port, Process 개념잡기

3. Process

3) 이전에 실행한 프로세스가 종료되지 않아 포트 충돌 나는 경우,
프로세스를 종료해서 해결

7. cron 개념잡기

1. cron

1) 컴퓨터 운영체제의 시간 기반 잡 스케줄러

: 소프트웨어 환경을 설정하고 관리하는 사람들이 작업을 고정된 시간, 날짜, 간격에 주기적으로 스케줄링하기 위해 사용

2) crontab 명령어

: `crontab -e` (명령을 등록, 편집)

: `crontab -d` (등록된 명령을 삭제)

: `crontab -l` (현재 등록된 리스트 출력)

: `crontab -l -u otheruser` (otheruser 사용자가 등록한 crontab 리스트 출력)

: `crontab -r` (현재 사용자가 등록한 crontab 전체 삭제)

7. cron 개념잡기

2. crontab 실습하기

1) 파이썬 코드 작성하기 (webapps - ch06 - cron - led.py)

```
1  import RPi.GPIO as GPIO
2  import time
3
4  GPIO.cleanup()
5
6  LED=8
7  GPIO.setmode(GPIO.BOARD)
8  GPIO.setup(LED, GPIO.OUT, initial=GPIO.LOW)
9
10 ~ try:
11     num = 0
12     while(True):
13         time.sleep(1)
14         GPIO.output(LED, GPIO.HIGH)
15         time.sleep(1)
16         GPIO.output(LED, GPIO.LOW)
17         num += 1
18         if(num == 5):
19             GPIO.cleanup()
20             break
21 ~ except KeyboardInterrupt:
22     GPIO.cleanup()
```

7. cron 개념잡기

2. crontab 실습하기

2) cron에 등록

: `$ crontab -e`

: crontab을 열고 가장 아래줄에 다음과 같이 적고 저장

`* * * * * sudo python /home/pi/webapps/ch06/cron/led.py`

: 편집을 빠져나오면 1분마다 5초씩 깜빡이는 LED 확인 가능

3) python코드에 한글이 있다면 첫번째 줄에 문자 인코딩을 선언

: `-*- encoding: utf-8 -*-`

`import RPi.GPIO as GPIO`

`import time`

~~ (생략)

7. cron 개념잡기

2. crontab 실습하기

4) crontab 설정규칙

: * * * * *
분(0~59) 시간(0~23) 일(1~31) 월(1~12) 요일(0~7)

5) crontab 설정 예

: * * * * * (매 1분마다 실행)
: */10 * * * * (매 10분마다 실행)
: 30 */6 * * * (매 6시간마다 실행, 00:30, 06:30, 12:30, 18:30)
: 45 5 * * 5 (매주 금요일 오전 5시 45분에 실행)

6) 더 이상 crontab 사용하지 않기

: \$ crontab -e (마지막에 적었던 코드를 삭제)
(* * * * * sudo python /home/pi/webapps/ch06/cron/
led.py)